21. Design Patterns in Modern AI Workflows: Factory, Strategy & Observer

Writing Industry-Standard Software Without Reinventing the Algorithmic Wheel
Module 4 • Difficulty Level: Intermediate / Advanced • Estimated Reading Time: 13 min

---

🎙️ NotebookLM & Audio Overview Podcast Briefing

Podcast Discussion Hook:
"The Gang of Four catalog was published in 1994; why are patterns like Strategy, Factory, and Observer still indispensable in modern AI-assisted engineering?"

3 Core Audio Takeaways:

- 📌 Design patterns are not academic dogma; they are a high-bandwidth shared vocabulary between humans and AI. - 📌 Instructing an AI model: 'Refactor this branching logic using the Strategy Pattern' conveys vast architectural nuance in a single sentence. - 📌 The Observer pattern is the architectural bedrock of modern reactive event-driven and streaming systems.

📖 1. The Story & Real-World Analogy

Visit a pizzeria and place an order.

Do you tell the server: *"Retrieve a porcelain dish from cupboard 3, measure 200 grams of flour, preheat the oven to 450 degrees, bake for 12 minutes, and carry it to my table"?* Never. You say: *"One Margherita pizza, please."* The kitchen encapsulates the instantiation mechanics. That is the **Factory Pattern**.

How do you settle the bill? Apple Pay, credit card, or cash? The cashier presents the appropriate terminal without altering the bill total. That is the **Strategy Pattern**.

When your pizza is boxed, a buzzer lights up on your table notifying you to pick it up. That is the **Observer Pattern**.

Design patterns are standard operational blueprints for everyday problems. The same rigor applies to scalable software.

🧠 2. The AI Mental Model & Paradigm Shift

The Essential Junior Pattern Triumvirate:

```
1. FACTORY PATTERN (Creational):
   Hides instantiation complexity. The caller asks: 'Give me a storage client for
AWS S3'.
   Use Cases: Multi-cloud adapters, database connection pooling.

2. STRATEGY PATTERN (Behavioral):
   Allows runtime interchangeability of algorithms without modifying the context
host.
   Use Cases: Dynamic discount policies, payment processing adapters.

3. OBSERVER PATTERN (Behavioral):
   Maintains a subscriber registry; state mutations trigger automatic updates to all
listeners.
   Use Cases: Event emitters, UI state subscription, webhook fan-out.
```

🤖 3. AI Prompt Lab (Production-Ready Templates)

You can directly paste and adapt this prompt template in Cursor, Claude Code, ChatGPT, or Copilot:

```
# ROLE: Software Design Patterns Architect
Our platform sends notifications across 3 distinct channels:
- SMS (Twilio)
- Email (SendGrid)
- Push Notifications (Firebase FCM)

## TASK:
Model this system in TypeScript using the Strategy Pattern:
1. Define a shared `NotificationStrategy` interface (`send(userId, message)`).
2. Implement discrete strategy classes for each provider.
3. Implement a `NotificationContext` class allowing dynamic runtime swapping of
notification channels.
```

💻 4. Code Walkthrough: Naive / Hallucination vs. Production-Grade

❌ Naive / Unchecked AI Output (Risky):

```
// Kötü: 20 tane if-else ile spagetti kod
export async function sendNotificationSpaghetti(type: string, userId: string, msg:
string) {
  if (type === "SMS") {
    // 30 satır Twilio kodu...
  } else if (type === "EMAIL") {
    // 30 satır SendGrid kodu...
  } else if (type === "PUSH") {
    // 30 satır Firebase kodu...
  } else {
    throw new Error("Bilinmeyen bildirim türü");
  }
}
```

✅ **Senior-Refined Version (Robust & Clean):**

```
// Strategy Pattern ile Zarif ve Genişletilebilir Mimari

// 1. Strateji Arayüzü (Contract)
export interface NotificationStrategy {
  send(userId: string, message: string): Promise<boolean>;
}

// 2. Somut Stratejiler (Concrete Strategies)
export class EmailNotificationStrategy implements NotificationStrategy {
  async send(userId: string, message: string): Promise<boolean> {
    console.log(`[SendGrid] ${userId} kullanıcısına e-posta gönderildi: ${message}
`);
    return true;
  }
}

export class SmsNotificationStrategy implements NotificationStrategy {
  async send(userId: string, message: string): Promise<boolean> {
    console.log(`[Twilio] ${userId} kullanıcısına SMS atıldı: ${message}`);
    return true;
  }
}

// 3. Bağlam Yöneticisi (Context)
export class NotificationService {
  private strategy: NotificationStrategy;

  constructor(initialStrategy: NotificationStrategy) {
    this.strategy = initialStrategy;
  }

  setStrategy(newStrategy: NotificationStrategy): void {
    this.strategy = newStrategy;
  }

  async dispatch(userId: string, message: string): Promise<boolean> {
    return await this.strategy.send(userId, message);
  }
}
```

💡 **Code Analysis:** Adding WhatsApp or Discord notifications requires zero modifications to existing classes (Open/Closed Principle). We simply implement a new `WhatsAppStrategy` conforming to the interface.

⚠️ 5. Junior Pitfalls & Senior Perspective

- **Trap:** Overusing patterns for simple binary conditions (e.g., boolean flags).

  - **Remedy:** A simple ternary or if/else is sufficient for binary branches; reserve patterns for extensible domains.
- **Trap:** Failing to unsubscribe listeners in the Observer pattern, causing memory leaks.
  - **Remedy:** Always clean up subscriptions when components unmount or sessions terminate.
- **Trap:** Concealing pattern intent with ambiguous class names.
  - **Remedy:** Explicitly append pattern suffixes: `StripePaymentStrategy`, `LoggerFactory`.

🎬 6. Udemy Video Lecture Blueprint & Slide Deck

📽️ Lecture Title: *Lecture 21: Design Patterns in Action - Factory, Strategy & Observer*

- 🖥️ **Slide 1:** Design patterns as the common vocabulary uniting engineers and AI models.
- 🖥️ **Slide 2:** Slaying the if-else branching beast with the Strategy Pattern.
- 🖥️ **Slide 3:** Decoupling instantiation complexity using the Factory Pattern.
- 🖥️ **Slide 4:** The Observer Pattern: Foundation of modern reactive and event-driven systems.

Live Coding / Screencast Scenario:
Implement a shipping cost calculator. Author 3 strategy classes for FedEx, UPS, and DHL. Demonstrate swapping strategies at runtime in response to user selection.

🎯 7. Daily Action Task & Mini Quiz

🛠️ Daily Action Task:

Refactor a multi-branch `switch` or `else-if` statement in your project into a polymorphic Strategy map.

🧠 Chapter Review Quiz:

Question 1: What is the primary architectural purpose of the Strategy Pattern? - A) Wiping database tables cleanly - B) Encapsulating a family of interchangeable algorithms to swap behaviors dynamically at runtime without modifying the client - C) Compressing HTML templates - D) Conserving laptop battery cycles > **Correct Answer:** B > **Explanation:** The Strategy Pattern enables dynamic runtime selection of algorithms by adhering to a shared polymorphic interface.

Question 2: In the Observer Pattern, what term designates the listening entities registered for notifications? - A) Subscribers / Observers - B) Compilers - C) Command Shells - D) Network Routers > **Correct Answer:** A > **Explanation:** The publisher maintains a registry of Observers/Subscribers and invokes their notification callback on state changes.